

Partial Class and Partial Methods in C# with Examples

In this article, I am going to discuss **Partial Class and Partial Methods in C#** with Examples. Please read our previous article, where we discussed [Method Hiding in C#](#). At the end of this article, you will understand what are Partial classes and Partial Methods and when and how to use Partial Classes and Partial Methods in C# with Examples.

What is Partial Class in C#?

Partial Class is the new feature that has been added in C# 2.0 which allows us to define a class on multiple files i.e. we can physically split the content of the class into different files but even physically they are divided but logically it is one single unit only. A class in which code can be written in two or more files is known as a partial class. To make any class partial we need to use the keyword `partial`.

According to Microsoft, It is possible to split the definition of a class over two or more source files. It is also possible to split the definition of a struct or an interface over two or more source files which we will discuss when we discuss the C# 8 new features. Each source file contains a section of the type i.e. class definition, and all parts are combined when the application is compiled.

Understanding Partial Class in C# with an Example:

Now, let us understand Partial Class with an example. First, create a console application and then add a class file with the name `Employee.cs`. Once you add the class file, then copy and paste the following code into it. Here, please notice the class name is `Employee` and in this class, we have declared 4 auto-implemented properties i.e. `FirstName`, `LastName`, `Gender`, and `Salary`. Here, we have also declared 2 public methods i.e. `DisplayFullName` and `DisplayEmployeeDetails`.

```
using System;
namespace PartialClassDemo
{
    public class Employee
    {
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public string Gender { get; set; }
        public double Salary { get; set; }

        public void DisplayFullName()
        {
            Console.WriteLine($"Full Name is : {FirstName} {LastName}");
        }
        public void DisplayEmployeeDetails()
        {
            Console.WriteLine("Employee Details : ");
            Console.WriteLine($"First Name : {FirstName}");
            Console.WriteLine($"Last Name : {LastName}");
            Console.WriteLine($"Gender : {Gender}");
            Console.WriteLine($"Salary : {Salary}");
        }
    }
}
```

```

    }
}

```

This is a very simple Employee class having 4 auto-implemented public properties, and 2 public methods. Now, let us consume the above class. Please modify the Program class which contains the Main method as shown below. Here, we are creating an instance of the Employee class and setting values for the four properties, and then invoking the DisplayFullName and DisplayEmployeeDetails method which is going to display the employee's full name and employee information on the console window.

```

using System;
namespace PartialClassDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Employee emp = new Employee
            {
                FirstName = "Pranaya",
                LastName = "Rout",
                Salary = 100000,
                Gender = "Male"
            };
            emp.DisplayFullName();
            emp.DisplayEmployeeDetails();
            Console.ReadKey();
        }
    }
}

```

Output:

```

Full Name is : Pranaya Rout
Employee Details :
First Name : Pranaya
Last Name : Rout
Gender : Male
Salary : 100000

```

Splitting Above Class Definition into 2 Files using Partial Classes in C#.

Now what we are going to do is, we will split the above Employee class definition into two different class files. One class file is going to contain all 4 public auto-implemented properties (FirstName, LastName, Gender, and Salary), and the other class file is going to contain the two public methods i.e. DisplayFullName and DisplayEmployeeDetails that we have defined inside the Employee class.

First, delete the Employee.cs class file from the project. Then we need to add two class files with the name PartialEmployeeOne and PartialEmployeeTwo. The PartialEmployeeOne class file going to contain all 4 public auto-implemented properties (FirstName, LastName, Gender, and Salary), and the PartialEmployeeTwo class file going to contain the two public DisplayFullName and DisplayEmployeeDetails

methods. Even though the class file names are different, the class name is going to be the same, and in this case, we are providing the class name as `PartialEmployee` in both the class file as well as we are making the class as partial by using the partial keyword.

Adding `PartialEmployeeOne.cs`

In order to add the `PartialEmployeeOne.cs` class file, right-click on the project and then add a class file with the name `PartialEmployeeOne.cs` and then copy and paste the following code into it. Notice that in the below code, the `PartialEmployee` class is marked with the partial keyword and it contains only the 4 public properties as per our requirements.

```
namespace PartialClassDemo
{
    public partial class PartialEmployee
    {
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public string Gender { get; set; }
        public double Salary { get; set; }
    }
}
```

Note: Here the class file name is `PartialEmployeeOne.cs` but the class name is `PartialEmployee` and the class is marked with the partial keyword which makes it a partial class.

Adding `PartialEmployeeTwo.cs`

Now let us add the `PartialEmployeeTwo.cs` class file to our project. To do so, right-click on the project and add a class file with the name `PartialEmployeeTwo.cs` and then copy and paste the following code into it. Notice that in the below code, the `PartialEmployee` class is also marked with the partial keyword and it contains only the two public methods as per our requirements. But the point that you need to observe is, here, we are able to access the properties that are defined in `PartialEmployeeOne.cs` class file and this is possible because of the partial class.

```
using System;
namespace PartialClassDemo
{
    public partial class PartialEmployee
    {
        public void DisplayFullName()
        {
            Console.WriteLine($"Full Name is : {FirstName} {LastName}");
        }
        public void DisplayEmployeeDetails()
        {
            Console.WriteLine("Employee Details : ");
            Console.WriteLine($"First Name : {FirstName}");
            Console.WriteLine($"Last Name : {LastName}");
            Console.WriteLine($"Gender : {Gender}");
            Console.WriteLine($"Salary : {Salary}");
        }
    }
}
```

```
}  
}
```

Here the class file name is PartialEmployeeTwo.cs but the class name is PartialEmployee and we also marked this class as partial by using the partial keyword. Even though the PartialEmployee class definition is split into multiple class files, after compilation, they will be grouped together and will become a single class only.

Using the Partial Class in C#:

Now, we are going to use the Partial classes whose definition is split across two different class files. We need to use the Partial class just like a normal class. We can create an instance and we can also invoke the members using the instance. So, modify the Main method of the Program class as shown below to use the PartialEmployee class. Here, you can see that we are creating an instance of the PartialEmployee class, setting the 4 property values, and then invoking the two public methods.

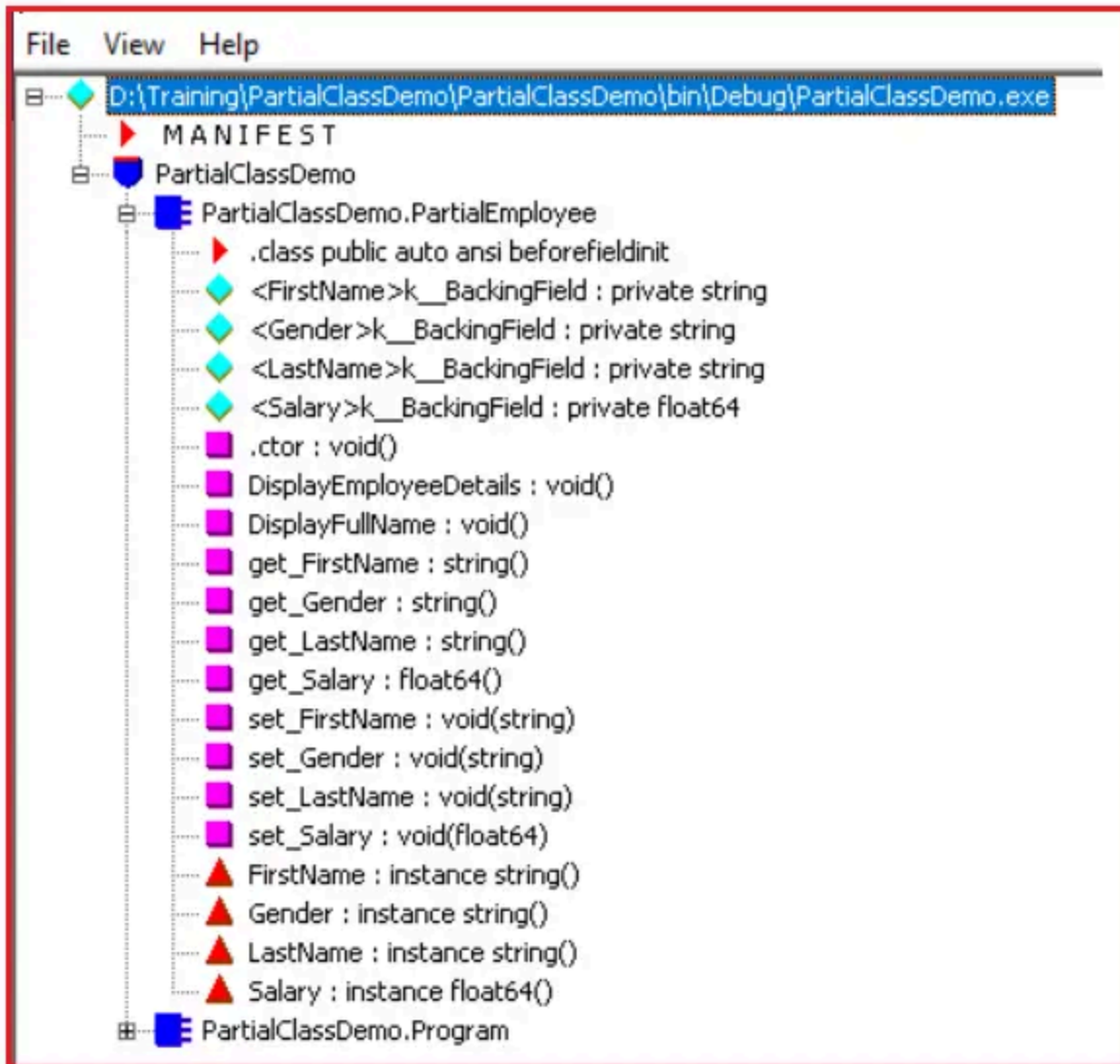
```
using System;  
namespace PartialClassDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            PartialEmployee emp = new PartialEmployee()  
            {  
                FirstName = "Pranaya",  
                LastName = "Rout",  
                Salary = 100000,  
                Gender = "Male"  
            };  
            emp.DisplayFullName();  
            emp.DisplayEmployeeDetails();  
            Console.ReadKey();  
        }  
    }  
}
```

Output:

```
Full Name is : Pranaya Rout  
Employee Details :  
First Name : Pranaya  
Last Name : Rout  
Gender : Male  
Salary : 100000
```

Now, you can see we are getting the same output as the previous example. Now, even though we have created two class files and split the class definition inside two class files, after compilation both the partial classes are combined into a single class. To prove this, open Visual Studio Developer Command Prompt and

then open the ILDASM tool and see the IL Code of the above program. Once you open the ILDASM code, then you will see that there is only one PartialEmployee class as shown in the below image.



So, the point that you need to remember is that even though the partial class definition splits into multiple files, after compilation they will become a single class.

When do we need to use Partial Class in C#?

There are several situations when splitting a class definition is desirable

1. When working on large projects, splitting a class over separate files allows multiple programmers to work on it simultaneously.
2. When working with automatically generated source code, the code can be added to the class without having to recreate the source file. Visual Studio uses this approach when creating windows form, Web service wrapper code, and so on.

In My Project, I am using Entity Framework Database First Approach. In that case, the Entity Framework will create the models i.e. the classes based on the database and it creates the classes as partial classes. Next, I want to do some modifications with the auto-generated partial classes like adding some additional properties or adding some attributes. But, if I do the modification with the auto-generated partial classes, then my

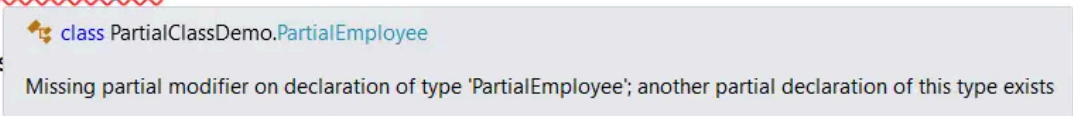
changes will be lost when I update the EDMX file. So, what I generally do is, create a partial class, and in that partial class, I do all the customization.

Rules to follow while working with Partial Classes in C#:

Rule1: All the parts that contain the class definition must use the partial keyword. All the parts must be available at compile time to form the final type. Otherwise, we will get a compilation error stating **Missing partial modifier. Another partial declaration of this type exists**. For a better understanding, please have a look at the below example code. Here, we can see that in this first definition we have used the partial keyword and but in the second definition of the class, we have not used the partial keyword, and hence the compiler throws one compilation error.

```
public partial class PartialEmployee{...}

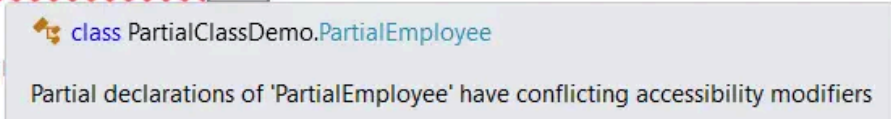
public class PartialEmployee
{
    public void DisplayEmployeeDetails(){...}
}
```



Rule2: All the parts of the partial class must have the same access modifier. If we try to use different access modifiers in different parts of the partial class, then we will get a compilation error saying that **Partial declarations have conflicting accessibility modifiers**. For a better understanding, please have a look at the below example code. Here, you can see that in the first definition of the PartialEmployee class we have used public access specifier, and in the second definition of the PartialEmployee class we have used internal access specifier and hence the compiler throws one compilation error.

```
public partial class PartialEmployee{...}

internal partial class PartialEmployee
{
    public void DisplayFullName(){...}
    public void DisplayEmployeeDetails(){...}
}
```



Rule3: If any of the parts are declared as abstract, then the entire type is considered as abstract or if any of the parts are declared as sealed, then the entire type is considered as sealed or if any of the parts inherit a class, then the entire type inherits that class. Here, you can see, one of the parts we declared as abstract, and hence the complete class becomes abstract and as we know we cannot create an instance of an abstract class. And here when we are trying to create an instance of the PartialEmployee class which is an abstract class, the compiler throws one compilation error.

```

public abstract partial class PartialEmployee...

public partial class PartialEmployee...

class Program
{
    static void Main(string[] args)
    {
        PartialEmployee obj = new PartialEmployee();
    }
}

```

 class PartialClassDemo.PartialEmployee

Cannot create an instance of the abstract class or interface 'PartialEmployee'

Rule4: C#.NET does not support multiple class inheritance that we already discussed in our [Multiple Inheritance](#) article because of the ambiguity problem. That means different parts of the partial class must not specify different base classes. If we specify different base classes, then we will get a compilation error saying **Partial declarations must not specify different base classes**. For a better understanding, please have a look at the below example code. Here, you can see that one part of the Partial Class inherits from the Employee base class and the other part of the Partial class inherits from the Customer base. That means now the Partial class has two base classes making this multiple inheritance with classes which is not possible in C# and hence the compiler throws one compilation error.

```

public class Employee
{
}

public class Customer
{
}

public partial class PartialClass : Employee
{
}

public partial class PartialClass : Customer
{
}

```

 class PartialClassDemo.PartialClass

Partial declarations of 'PartialClass' must not specify different base classes

Rule5: But it is possible with Partial classes that the different parts of the partial class can specify different base interfaces and the final type should and must implement all the interface methods. In the following example, the PartialClass needs to provide the implementation for both IEmployee and ICustomer interface methods. This is possible because a class can not have more than one base class, but a class can have more than one base interface and this is how we can implement multiple inheritances in C# with interfaces.

```

namespace PartialClassDemo
{
    public interface IEmployee

```

```

{
    void EmployeeMethod();
}
public interface ICustomer
{
    void CustomerMethod();
}
public partial class PartialClass : IEmployee
{
    public void EmployeeMethod()
    {
        //Method Implementation
    }
}
public partial class PartialClass : ICustomer
{
    public void CustomerMethod()
    {
        //Method Implementation
    }
}
}

```

Note: Any members that are declared in a partial definition are available to all of the other parts of the partial class. Once we understand Partial Classes, now let's move further and try to understand Partial Methods in C#.

What are Partial Methods in C#?

A partial class may contain a partial method. One part of the class contains the signature of the method. An implementation of the Partial Method can be defined in the same part or other parts of the Partial Class. If the implementation is not supplied, then the method and all calls to the partial method are removed by the compiler at the time of compilation.

Let us understand partial methods with an example. Create a console application. Add a class file with the name PartialClassOne.cs to the project and then copy and paste the following code into it.

```

using System;
namespace PartialClassDemo
{
    partial class PartialClass
    {
        // Declaration of the partial method.
        partial void PartialMethod();
        // A public method calling the partial method
        public void PublicMethod()
        {
            Console.WriteLine("Public Method Invoked");
            PartialMethod();
        }
    }
}

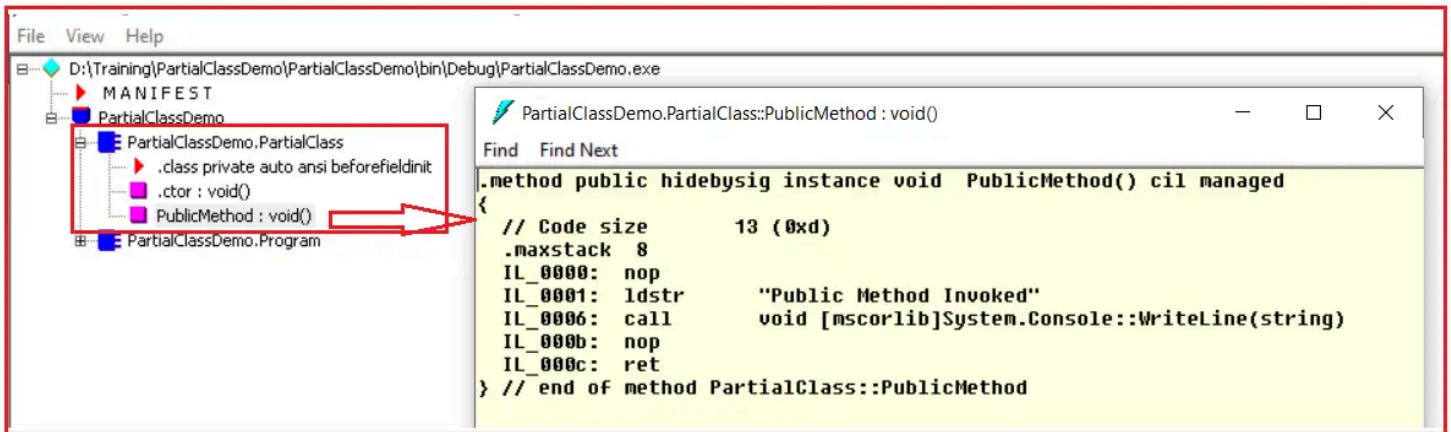
```

```

    }
}

```

As you can see in the above code, we have created the `PartialMethod()` using the `partial` keyword which makes it a partial method, and further notice this partial method does not have its body, it contains only the method signature. The implementation of this Partial you can provide in this class definition or you can also provide the implementation in other parts of this partial class. But the most important point that you need to remember is that the implementation of a partial method is optional. If we don't provide the implementation, then the method and all calls to the partial method are simply removed by the compiler at the time of compilation. Now, if you build the application and if you see the IL code using the ILDASM tool, then you will see that the Partial method is removed from the Partial class as well as the call to the partial method from the public method is also removed as shown in the below image.



In this example, the partial `PartialMethod()` is invoked within the `PublicMethod()` but we don't have the partial method implementation. To prove this, copy and paste the following code in the `Main()` method of the `Program` class. Here, we are creating an instance of the `Partial Class` and invoking the public method.

```

using System;
namespace PartialClassDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            PartialClass SC = new PartialClass();
            SC.PublicMethod();
            Console.ReadKey();
        }
    }
}

```

Output: Public Method Invoked

When we run the above application, notice that we don't get a compiler error, in spite of not having an implementation for the partial `PartialMethod()`. Since the implementation of the partial method is missing, the compiler will remove the signature and all calls to the method.

The implementation of the Partial Methods as discussed can be provided in the same part or in other parts of the partial class. Now, let us add a class file with the name `PartialClassTwo.cs` and then copy and paste the

following code into it. The implementation of the partial method is provided here.

```
using System;
namespace PartialClassDemo
{
    partial class PartialClass
    {
        // Partial method implemented
        partial void PartialMethod()
        {
            Console.WriteLine("Partial PartialMethod  Invoked");
        }
    }
}
```

Now, run the console application again, and please observe the output. This time, both the partial method and the public method messages are printed on the console as shown below.

```
Public Method Invoked
Partial PartialMethod  Invoked
```

Rules to Follow while working with Partial Methods in C#:

While working with Partial Methods in C#, we need to follow some rules and regulations. Let us understand all those rules and regulations one by one with examples.

Rule1: Partial methods in C# are private by default and we can use any access specifier explicitly, and if we try to use any access specifier explicitly like public, private, protected, etc, then we will get a compiler stating **A partial method cannot have access modifiers or the virtual, abstract, override, new, sealed, or extern modifiers**. For a better understanding, please have a look at the below example code. Here, we are explicitly trying to use a private access modifier with the partial method, and hence the compiler throws an error.

```
partial class PartialClass
{
    private partial void PartialMethod();
}
```

 void PartialClass.PartialMethod()

A partial method cannot have access modifiers or the virtual, abstract, override, new, sealed, or extern modifiers

Rule2: Partial method declaration and implementation should not be at the same time. Its declaration should be at one place and its implementation must be at a different place either in the same part or in other parts of the partial class. If we include both declaration and implementation at the same time, then we will get a compiler error saying **No defining declaration found for implementing declaration of partial method 'PartialDemo.PartialClass.partialMethod()'**. For a better understanding, please have a look at the below example code. Here, we are declaring and implementing the partial method at the same time and hence we are getting a compiler error.

```
partial class PartialClass
```

```
{  
    partial void PartialMethod()  
    {  
        Console.WriteLine()  
    }  
}
```

 void PartialClass.PartialMethod()

No defining declaration found for implementing declaration of partial method 'PartialClass.PartialMethod()'

Rule3: A partial method return type must be void. If we try to use any other return type like int, string, bool, etc. then we will get a compiler error saying **Partial methods must have a void return type**. For a better understanding, please have a look at the below example code. Here, we are trying to make the return type as int and hence the compiler throws an error.

```
partial class PartialClass
```

```
{  
    partial int PartialMethod();  
}
```

 int PartialClass.PartialMethod()

Partial methods must have a void return type

Rule4: A partial method must and should be declared within a partial class or partial struct. A non-partial class or struct cannot include partial methods. If we try to define a partial method under a non-partial class or struct, then we will get a compile-time error saying **A partial method must be declared within a partial class or partial struct**. For a better understanding, please have a look at the below code. Here, we are trying to define a partial method under a non-partial class and hence the compiler throws an error.

```
public class PartialClass
```

```
{  
    partial void PartialMethod();  
}
```

 void PartialClass.PartialMethod()

A partial method must be declared within a partial class or partial struct

Rule5: The signature of the partial method declaration must match the signature of the implementation else we will get a compilation error. For a better understanding, please have a look at the following code. Here, in the first part of the partial class, we have defined the partial method with one integer parameter and in the second part of the partial class where we are providing the implementation, we are not including the integer parameter and hence the compiler throws an error.

```

public partial class PartialClass
{
    partial void PartialMethod(int x);
}

public partial class PartialClass
{
    //Method Signature not Matched
    partial void PartialMethod()
    {
        //Implementation
    }
}

```

Rule6: A partial method can be implemented only once either in the same part or in other parts of the partial class. If we try to implement a partial method more than once, then we will get a compilation error saying **A partial method may not have multiple implementing declarations**. For a better understanding, please have a look at the below example code. Here, in the first part, we have declared the partial method and in the second part and in the part we are providing implementation to the partial method, and as we are providing two implementations we are getting the compilation error. This is because, in a class, we cannot define two methods with the same name and same parameters i.e. method overriding is not possible in a single class.

```

public partial class PartialClass
{
    partial void PartialMethod();
}

public partial class PartialClass
{
    partial void PartialMethod()
    {
        //Implementation one
    }
}

public partial class PartialClass
{
    partial void PartialMethod()
    {
        //Implementa
    }
}

```

 void PartialClass.PartialMethod()

A partial method may not have multiple implementing declarations

In the next article, I am going to discuss **Sealed Class in C#** with Examples. Here, in this article, I try to explain **Partial Class and Methods in C#** with Examples. I hope this Partial Class and Partial Methods in C#

with Examples article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about these Partial Classes and Partial Methods in the C# article.